

# The Noisy Neighbour Is a Robot: Characterizing Multi-Agent Workloads on the Open Lakehouse

**Target:** VLDB 2027 Industrial Track (deadline 2 Mar 2027 — verify)

**Status:** Draft v0.4 — Sections 1–10 complete; related work read from released artifacts

**Artifact:** `harness/` in this repo (`agentlake`), 27 tests passing

**Author:** Kiran Bhusnurmath

**Source status.** Four of the five works this paper positions against were read from their released artifacts: SQLStorm [1] (repository, query sets and documentation), AgenticDataBench [3] (repository; the skill, task and domain counts below were re-computed from its data files), FDABench [4] (repository, technical report and agent source, from which the phase-instrumentation description is taken) and the data-agents survey [22] (full PDF; the term-frequency table in Section 2.3 is computed over its text). Text-to-Big SQL [2] has no released artifact we could locate and is characterized from its abstract and published summaries; Section 2.3 stays within what those support, and its specific figures are attributed rather than asserted. Confirm [2] against its full text before submission.

All quantitative results are measured by the released harness and reproducible with the commands in `harness/README.md`. Claims that would require production telemetry are not made; Section 8 says where such data would have helped.

---

## Abstract

Analytical platforms were designed for a consumer that no longer dominates their workload. A human analyst issues few, deliberate queries and reuses saved logic. An LLM agent samples schemas, generates several equivalent candidates per question, retries, and abandons partial work. Enterprises now point fleets of them at a shared lakehouse.

The evaluation literature has followed the agent, not the platform. Data-agent benchmarks decompose an agent's latency and tokens by phase, including retries; recent text-to-SQL work shows execution overtaking agent interaction as data grows. All of it accounts for what one agent spends on one task. We hold data fixed, grow the *fleet*, and account for what the *platform* spends, via `agentlake`, an open harness instrumenting scan cost, cache behaviour, catalog traffic and contention rather than accuracy.

Three results. Per-query monitoring inverts the sign of the cost result: agent queries scan 32% fewer rows each than a human-analyst control while costing 2.58x more per answered question, so existing instrumentation reports an improvement where there is a regression. The workload is additive in work but superlinear in latency: scan cost per answered question is flat in fleet size ( $k = -0.02$ ) while mean latency scales as  $\text{fleet}^{0.76}$  and throughput saturates at four agents. Of five mitigations, the best is the least discussed — cached column profiles cut 35.9% of scan cost with no loss of answers — while tightening correction budgets, a common reflex, costs 3.4% *more* per answer.

We release the harness, with the negative results and the artifacts we corrected.

# 1. Introduction

The open lakehouse settled its architectural argument faster than most people expected. By 2026 the table format question is effectively closed — Apache Iceberg v3 reached general availability across the major platforms, and the REST catalog became the interoperability point that lets independent engines discover tables, resolve metadata, coordinate atomic commits, and enforce access over a single copy of data. The industry spent fifteen years arguing about storage formats and metadata layers, and won.

It won just in time to face a consumer it was not designed for.

The design centre of every analytical engine in production today is a human being with a question. That consumer has properties the whole stack quietly depends on. They issue few queries relative to the thinking time between them. They reuse logic — saved queries, dashboards, dbt models, semantic definitions — so the same shapes recur and caches work. They possess tacit knowledge of the schema and do not need to rediscover it. They exercise judgement about cost: an analyst who watches a query scan a petabyte will usually add a partition filter before running it again. And critically, their access pattern is stable enough to be audited, which is the assumption underneath every row- and column-level security model in the field.

An LLM agent violates all five.

A single user question handed to an agent does not become a single query. It becomes a burst: metadata introspection to find candidate tables, sampling to understand column semantics and cardinality, one or more candidate queries — often several semantically equivalent formulations, which in a Big Data engine can differ enormously in physical plan quality, shuffle volume and scan efficiency at scale — then self-correction retries when a query errors or the agent's own confidence check fails, and abandoned work when the plan changes mid-task. The agent has no tacit schema knowledge; it rediscovers context on every cold start. It has no felt sense of cost. And its access pattern is generated at inference time from an input that may be adversarial.

One agent behaving this way is a curiosity. The situation now arriving in enterprises is different: fleets of agents — embedded in applications, invoked by other agents, running on schedules — sharing one governed lakehouse. We do not have a production census of that fleet size, and we are careful not to assert one: the case this paper makes does not depend on how many agents any particular organisation is running today, only on the fact that the number is no longer one and that the platform cannot tell the difference.

The research community has moved quickly on the agent side of this interface and barely at all on the platform side. A substantial benchmark literature asks whether an agent produces *correct* SQL; recent work has extended that to partial correctness and cost at scale; data-agent benchmarks now decompose an agent's own latency and token spend by phase; and a parallel strand uses LLM-generated workloads to benchmark engines more broadly than TPC-H allows (Section 2.3). This is real progress, and all of it evaluates one agent against one task, accounting for what that agent spent. None of it runs a fleet against a shared system and accounts for what the system spent — what a hundred agents do to a shared result cache, a shared catalog, a shared executor pool, or a shared cost centre.

That gap is the subject of this paper. Our contributions:

1. **A workload characterization of multi-agent analytical traffic** against an open Iceberg lakehouse, along platform-side axes that the accuracy-oriented literature does not instrument: scan amplification, cache behaviour under near-duplicate queries, catalog request distribution, and cost per *answered question* rather than per *issued query* (Section 3).

2. **A taxonomy of agent-originated query traffic** that separates the phases of an agent's turn — discovery, sampling, candidate generation and correction, with abandonment as a disposition orthogonal to phase — and shows that these phases have sharply different platform cost profiles, which matters because they are conventionally measured as one undifferentiated workload (Section 3.1).
3. **An account of three structural mismatches** between lakehouse design assumptions and agentic consumption: workload shape, governance under non-deterministic consumers, and colocated agent write state (Section 4).
4. **A controlled ablation of five mitigations**, measuring what each recovers and what it costs in capability — including the finding that the most-discussed intervention saves no scan work and that a common throttling reflex is counterproductive (Section 6).
5. **agentlake**, an open harness for reproducing multi-agent platform load, released with this paper (Section 5). It runs in-process with no cluster, API key or network access, so every number reported here is reproducible on a laptop.

We deliberately do not contribute another accuracy benchmark. The question of whether an agent gets the answer right is well covered. The question of whether the platform survives a thousand agents trying is not.

---

## 2. Background

### 2.1 The open lakehouse as it now stands

The architecture this paper stresses has converged. Table format and metadata layer are settled questions, and the REST catalog has become the coordination point through which independent engines discover tables, resolve metadata, coordinate atomic commits and enforce access over a single physical copy of the data. Multi-engine access to one copy is the property that makes the lakehouse worth defending, and it is also what makes a new class of consumer a platform-wide concern rather than a per-tool one.

Four assumptions in that design matter for what follows, and none of them is stated anywhere as an assumption because until recently none of them was contentious.

**Metadata is a shared service with its own request pattern.** The catalog is consulted far more often than it is modified, and its load is dominated by a modest number of long-lived engines resolving a stable working set of tables. It is sized for that.

**Result caches are keyed on query identity.** An engine that has computed an answer will serve it again if it recognises the question, and recognition is a syntactic test against query text or a normalisation of it. This works because the queries that repeat are generated by dashboards and saved reports, which emit character-identical SQL.

**Physical layout assumes a recurring predicate distribution.** Partitioning, clustering and file pruning all pay off in proportion to how well the predicates in the workload match the layout chosen for it. That match is maintained by observing which predicates recur.

**Access control binds to principal identity.** Row- and column-level policies are evaluated against who is asking. The implicit premise is that a principal's plausible access pattern is bounded by their role, so unusual access is detectable as an anomaly.

Field accounts of how lakehouses and data lakes actually behave in practice [10]-[12] make clear that these assumptions were already under strain before agents arrived.

Each of these holds for a consumer that is deliberate, repetitive and auditable. Section 4 argues that agent fleets are none of those, and Section 3 measures what that costs.

## 2.2 The agentic consumer

An analytical agent answers a question through a tool-calling loop rather than a single translation step. Given a question in natural language it must establish what data exists, what the columns mean, which tables join to which, and what the business definition of the requested measure is — none of which it holds a priori — before it can emit a query at all. Having emitted one, it inspects the result and may reject its own work: an error, an empty result set, or a value that fails an internal plausibility check all send it back around the loop.

Two consequences follow, and they are the ones that matter for the platform.

The first is that the unit of work is not the query. A user asks one question; the platform sees a burst. Section 3.1 decomposes that burst, but the important structural point is that the mapping from questions to queries is many-to-one, variable, and not observable from inside the engine. The engine sees queries arriving from a principal. It cannot see which of them belong to the same question, which were superseded, or which were discarded.

The second is that the interface between agent and data is itself under active redesign, in a direction that is worth noting because it is a response to failures in the field rather than a research programme. Pointing an agent at raw tables makes it re-derive joins, grain and metric definitions on every prompt, which produces inconsistent answers to the same question across runs. The response converging in practice is to stop exposing tables and expose a governed semantic model instead — measures and dimensions with declared definitions — which the agent introspects and calls through a protocol layer; work on natural-language access to Iceberg tables via LLM agents [13] and on agent communication protocols [19] describes the same interface from the agent's side. That several independent vendors arrived at the same shape within roughly a year is a signal about the inadequacy of the raw-table interface, not an endorsement of any particular implementation. Section 4.2 examines what that boundary does and does not provide, and Section 6.4 measures what it costs.

## 2.3 What the existing literature measures

Work at the intersection of LLMs and data systems has grown quickly, and it divides into three strands, none of which measures what happens to a platform serving many agents.

**Benchmarks of engines, using generated workloads.** SQLStorm [1] uses an LLM as a query *generator* rather than as a subject: GPT-4o-mini produces analytical queries against a schema, and the resulting workload is used to benchmark database systems. The released benchmark contains 18,251 queries on a StackOverflow dataset at 1GB, 12GB and 222GB, plus comparable sets on TPC-H, TPC-DS and JOB, run across eight engines with a per-query timeout. Its argument is that human-written benchmarks such as TPC-H, TPC-DS and JOB cover a narrow band of SQL functionality, and that generated workloads cover far more. This is the closest relative to our work in spirit — both use generated workloads to stress engines rather than to score models — and the difference is precisely our subject: SQLStorm issues one query at a time, with no agent loop around it, so it has no notion of discovery, sampling, correction, abandonment, fan-out or concurrency. It is worth noting one of its incidental findings, that LLM-generated SQL skews toward PostgreSQL dialect and required an explicit rewriting pass for cross-engine compatibility; dialect bias is a property of this consumer that a platform serving several engines will meet.

**Evaluations of text-to-SQL translation.** A large literature scores whether generated SQL is correct [5]-[9]. The most relevant recent development is a move past binary correctness: text-to-Big SQL [2] argues that a 0/1 label is the wrong instrument at scale, since an unnecessarily projected column is not equivalent to a wrong answer when

re-running the query is expensive, and proposes metrics that account jointly for partial correctness and cost. Its "both ends" framing is the part most relevant to us. Evaluating against TPC-H across scale factors, it reports that at small scale the agent's own interaction dominates total response time, while at large scale query execution dominates — TPC-H Q21's execution time grows 13.3x between SF 10 and SF 1000. Both ends, in their phrase, can break interactive analytics.

This is complementary to our Finding 4 and helps locate it. They hold the fleet at one agent and grow the *data*, finding execution time taking over as scale rises; we hold the data fixed and grow the *fleet*, and find execution latency taking over again, this time through queueing rather than volume. Their result says the execution side dominates at scale; ours says it degrades superlinearly once the execution side is shared.

**Benchmarks of data agents.** A cluster of recent benchmarks evaluates agents on multi-step analytical tasks rather than single translations, and two of them are close enough to this work to require care.

FDABench [4] (KDD 2026) contributes 2,007 tasks over genuinely heterogeneous sources — relational databases including SQLite, BigQuery and Snowflake, alongside PDF, video, audio and web content — in single-choice, multiple-choice and report formats, generated through an agent-expert collaboration framework. Two of its properties matter here, and we state them accurately rather than conveniently. First, it ships several agent architectures including a *multi-agent* one; that architecture is a coordinator directing specialist expert agents at a single task, not a fleet of independent agents sharing a platform. Second, and more relevant, its instrumentation decomposes agent latency and token usage by phase, reporting `decision`, `execute`, `retry` and `generate` ratios per task. That is a phase decomposition with an explicit retry term, and it is the closest existing construct to the taxonomy in Section 3.1. The difference is which side of the interface it measures: FDABench accounts for *time and tokens spent by the agent*, per task; we account for *rows scanned and cache behaviour on the platform*, attributed to the same phases and aggregated across a fleet. They are complementary and neither substitutes for the other.

AgenticDataBench [3] takes a different and instructive approach to coverage: it mines 6,510 StackOverflow data-science solutions into 433 clustered skills and composes those into 344 tasks across 15 domains (246 public, 98 withheld as a private test set). Grading is skill-level rather than aggregate, so failures can be localised rather than merely counted, and its leaderboard reports token cost alongside score across several agent harnesses. Like FDABench, its cost accounting is agent-side.

Others in this family include DataSpace, DAComp, KramaBench, CoDA-Bench and DABstep [21].

**The gap, quantified.** Our claim that the platform side is unexamined is not an impression. A recent 31-page survey of data agents [22], covering roughly 200 references and proposing an L0–L5 autonomy taxonomy for the field, offers a way to check it. Across its 28,688 words:

| Platform-side term        | Occurrences | Agent-side term | Occurrences |
|---------------------------|-------------|-----------------|-------------|
| throughput                | 0           | LLM             | 178         |
| queue                     | 0           | tool            | 61          |
| contention                | 0           | planning        | 24          |
| concurrent execution      | 0           | memory          | 15          |
| result cache / cache hit  | 0           | benchmark       | 12          |
| scan                      | 0           | accuracy        | 8           |
| fan-out                   | 0           |                 |             |
| retry / abandon           | 0           |                 |             |
| admission control         | 0           |                 |             |
| workload characterization | 0           |                 |             |
| lakehouse / Iceberg       | 0           |                 |             |
| catalog                   | 1           |                 |             |

The survey is a good survey and this is not a criticism of it. It accurately reflects where the field's attention sits. The vocabulary in which a platform engineer would describe serving an agent fleet — throughput, queueing, contention, cache behaviour, scan cost, admission control — does not appear in the field's own comprehensive account of itself.

We state the resulting claim precisely, because two of the benchmarks above come closer than a keyword count alone would suggest. FDABench does decompose cost by phase and does include a retry term; AgenticDataBench does report cost alongside accuracy. What none of this work does — and what the survey's vocabulary confirms is not yet a topic — is measure the *platform*: rows scanned, cache behaviour under near-duplicate queries, catalog request distribution, or what happens when more than one agent runs at a time. Every evaluation above runs one agent against one task and accounts for what that agent spent. None runs a fleet against a shared system and accounts for what the system spent.

A second scoping consequence is easy to miss. An evaluation scoring the *final answer* is structurally unable to see the work discarded on the way to it. Sampling queries, superseded corrections and abandoned turns are invisible to any metric computed on the answer, and Section 3.3 finds they account for roughly 60% of the rows the platform scans. FDABench's retry ratio is the partial exception that proves the rule: it can see that retries took the agent time, but not what they cost the engine underneath.

We therefore position this work as complementary rather than competing. We contribute no accuracy result and no text-to-SQL technique. We measure the platform.

### 3. Characterizing the workload

#### 3.1 A phase taxonomy of the agent turn

We decompose a single agent turn into four phases, each with a distinct platform cost signature, plus one disposition that can apply to any of them:

| Phase                | What the agent is doing                                                    | Platform cost signature                                                                                        |
|----------------------|----------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| Discovery            | Enumerating namespaces, tables, schemas                                    | Catalog metadata requests; negligible scan; high request count, low bytes                                      |
| Sampling             | LIMIT ed reads to learn column semantics, cardinality, value distributions | Small scans, poor selectivity, frequently unpartitioned; high file-open count relative to bytes returned       |
| Candidate generation | Issuing one or more full analytical queries                                | The only phase resembling a conventional BI workload                                                           |
| Correction           | Re-issuing after an error, an empty result, or a failed self-check         | Near-duplicate of the prior query; defeats exact-match result caching while duplicating nearly all of its work |

**Abandonment is a disposition, not a phase.** Our first model treated it as a fifth phase, and implementing it showed why that is wrong: a discovery query and a candidate query can both be abandoned, so relabelling abandoned work as "abandoned" destroyed the per-phase cost attribution the taxonomy exists to provide. Abandonment is therefore carried as a flag orthogonal to phase, and abandoned work retains the phase that produced it. We report the correction because the distinction is easy to miss on paper and unavoidable in code.

The taxonomy matters because these phases are conventionally aggregated into one number — "agent query volume" — and that number misleads in both directions. Discovery and sampling are cheap in bytes but expensive in request count, which is the load profile that stresses a catalog rather than an engine. Correction and abandoned work are expensive in bytes and invisible to metrics computed on the final answer.

The closest existing construct is FDABench's decomposition of *agent* latency and tokens into decision, execute, retry and generate phases (Section 2.3). The phases are recognisably similar; what differs is the quantity attributed to them. FDABench asks how long the agent spent in each; we ask what each cost the engine underneath.

In the reference configuration of Section 3.3, sampling accounts for the largest share of rows scanned despite being the phase that looks cheapest per query, and abandoned work for 7.6%. Both figures are properties of our phase model rather than of any deployment; what transfers is the ordering, not the magnitude.

#### 3.2 Measurement methodology

We release `agentlake`, an open harness that drives a configurable fleet of analytical agents against a lakehouse under test and instruments the platform side. It runs in-process on DuckDB over a natively generated star schema, with no cluster, no API key and no network access, so any number reported here can be reproduced on a laptop.

**Agents are synthetic.** No model is in the loop. A turn is generated from an explicit phase model — counts for discovery and sampling, a correction probability, an abandonment probability — and is fully determined by a seed. This is a deliberate trade: we give up the realism of a live agent's decisions in exchange for reproducibility and for the ability to vary one parameter at a time. The phase model reproduces the *shape* of agent traffic, not the judgement of any particular agent.

**Every phase parameter is an assumption, not a measurement.** The harness records this explicitly: each `PhaseModel` carries a `provenance` field, and the defaults declare themselves uncalibrated. All results below were produced with uncalibrated defaults and should be read as characterizing the *structure* of the cost, not its magnitude in any real deployment. Recalibration against production traces is the single change that would most strengthen this work, and Section 7 argues for the trace format that would enable it.

**Corrections are constructed, not hoped for.** To ask whether a result cache catches correction-phase queries, we need correction queries that are semantically equivalent to what they replace. The harness rewrites a canonical query into equivalent forms — alias renaming, comma-join, CTE wrapping, tautological predicates, reformatting — and the test suite asserts that every rewrite returns identical rows. Without that assertion, any cache finding would be an artifact of the mutator emitting genuinely different queries.

**Two caches are simulated over one query stream.** `ExactTextCache` keys on normalised query text, as analytical engines conventionally do. `PlanCache` keys on a fingerprint of the optimised operator tree. Simulating both over the identical stream provides the counterfactual that a real engine, which gives you one cache, cannot.

The fingerprint strips aliases, whitespace, case and cardinality estimates but retains filter literals. This matters more than it sounds: an earlier, coarser fingerprint collided `revenue by region` with `units by store`, which would have made the plan cache serve wrong results and inflated its measured hit rate. The test suite now checks both directions — equivalent rewrites must collide, and questions differing only in a filter literal must not.

**The sweep holds total work constant.** Fleet size varies; total turns across the fleet does not. Our first attempt held turns *per agent* constant instead, which made larger fleets do proportionally more total work, warmed the shared cache further, and produced a spurious *negative* scaling exponent — per-question cost appeared to fall with fleet size for reasons that had nothing to do with concurrency. Questions are drawn Zipf-style from a parameterised space so that question diversity is a property of the workload rather than an accident of fleet size.

### 3.3 Findings

Reference configuration: fleet sizes 1, 2, 4, 8, 16 and 32; 192 total turns held constant across sizes; 400,000 fact rows; uncalibrated default phase model. Each fleet size gets a fresh engine and fresh caches. The control condition is an idealised human analyst — one deliberate query per question, no rediscovery, no retry, no abandonment.

|                                    | agentic | human control | ratio        |
|------------------------------------|---------|---------------|--------------|
| Queries per answered question      | 3.81    | 1.00          | 3.8x         |
| Rows scanned per answered question | 969,865 | 375,731       | <b>2.58x</b> |
| Rows scanned per issued query      | 254,416 | 375,731       | <b>0.68x</b> |
| Scanned rows never delivered       | 60.7%   | 0.0%          | —            |
| Rows scanned on abandoned turns    | 7.6%    | 0.0%          | —            |

**Finding 1: per-query monitoring inverts the sign of the result.** Agent-issued queries scan 32% *fewer* rows each than the control, while the workload costs 2.58x more per unit of delivered value. A conventional per-query dashboard would report that the platform had become more efficient. The discrepancy is exactly the fan-out factor: the

denominator changed and the metric did not follow. We consider this the paper’s most practically consequential result, because it means the instrumentation most organizations already have will mislead them in the reassuring direction.

**Finding 2: roughly 60% of scanned rows never reach a user.** Sampling, superseded corrections and abandoned turns account for the majority of platform work. Abandoned work alone accounts for about 7.6%, and it is invisible to any metric computed on the final answer, because a benchmark that scores the answer cannot see what was discarded on the way to it. An agent-side phase accounting such as FDABench’s would see the *time* such work took; what neither sees without platform instrumentation is the scan it caused.

**Finding 3: plan-keyed caching closes about 41% of the residual misses** left by exact-text caching (93.4% vs 88.8% hit rate). The gap is the correction phase: rewrites that are semantically identical and textually distinct. This is a concrete, actionable result for an engine team, and it does not require any change to how agents behave.

**Finding 4: additive in work, superlinear in latency.** This finding arrived in two stages and the first stage was wrong in an instructive way.

Under the serial driver — which interleaves agents deterministically but executes their queries one at a time — rows scanned per answered question is flat in fleet size from 1 to 32 agents, with a fitted exponent of  $k = -0.026$  against  $k = 0$  for exact linearity. Read alone, that says agent cost is additive: a fleet creates no more work than the sum of its members.

That reading is incomplete, because the serial driver measures the work a fleet *creates* and not the contention it *causes*. We re-ran the sweep through a thread pool against a shared database with a deliberately bounded budget — four worker threads and 512MB — with one cursor per thread, so queries execute genuinely in parallel and genuinely compete.

| Fleet | Scan/question | Mean latency | p95 latency | Throughput     |
|-------|---------------|--------------|-------------|----------------|
| 1     | 998,791       | 3.45 ms      | 6.73 ms     | 172 q/s        |
| 2     | 925,818       | 3.72 ms      | 7.48 ms     | 314 q/s        |
| 4     | 869,869       | 5.84 ms      | 12.41 ms    | <b>364 q/s</b> |
| 8     | 909,145       | 12.32 ms     | 21.81 ms    | 309 q/s        |
| 16    | 928,427       | 22.61 ms     | 39.71 ms    | 299 q/s        |
| 32    | 885,560       | 39.45 ms     | 67.56 ms    | 326 q/s        |

  

| Metric                             | Scaling exponent $k$ |
|------------------------------------|----------------------|
| Rows scanned per answered question | <b>-0.023</b>        |
| Mean query latency                 | <b>+0.756</b>        |
| p95 query latency                  | <b>+0.705</b>        |

The work result survives concurrency unchanged:  $k = -0.023$ , still flat. Latency does not. Mean latency scales as approximately  $\text{fleet}^{0.76}$ , rising 11.4x from one agent to thirty-two, and p95 tracks it at  $\text{fleet}^{0.70}$ . Throughput peaks at four concurrent agents and is flat thereafter.

The interpretation is that beyond saturation a fleet adds no work and no throughput — it adds queue. This matters for how the problem should be framed. An organisation reasoning about agent adoption in terms of total compute consumed will conclude, correctly, that cost grows linearly with fleet size. An organisation reasoning about whether agents remain usable will find that interactive latency degrades superlinearly well before any cost signal moves. Those two conclusions are both true and they point to different interventions: the first is a budgeting problem, the second an admission-control problem.

We flag the residual caveat plainly. This is an in-process engine with a shared buffer pool and worker pool but no network shuffle, no distributed catalog under load and no cross-node scheduling. We now have evidence that contention effects appear as soon as a real shared resource pool exists, which was the substance of our original concern; we do not have evidence about the magnitude on a distributed engine, where a loaded REST catalog and shuffle are additional contention points that could plausibly make it worse. Section 7 lists that replication as the first open problem.

**Catalog skew** sits at 0.27–0.31 (Gini) and is stable across fleet sizes. We note it without drawing a conclusion: skew here is a product of our question-driven discovery model, and while an earlier uniform-draw model made skew unmeasurable by construction, the present value still reflects modelling choices more than platform behaviour.

## 4. Three structural mismatches

### 4.1 Workload shape

The mechanisms an analytical engine uses to be fast — caching, physical layout, plan reuse — all convert *recurrence* into speed. They are, in effect, bets that the next query will resemble the last one. Agent traffic settles those bets badly, and in two opposite directions at once.

At one end there is a head of near-duplicates that the engine cannot recognise as such. Correction-phase queries are semantically identical to the queries they replace and textually distinct from them, because a model regenerating a query does not reproduce its own formatting, aliasing or clause ordering. A text-keyed result cache treats each as new work. Section 3.3 measures the gap: plan-keyed caching recovers 8.0% more scanned rows than text-keyed, and the entire difference is this head. The engine is doing work it has already done and cannot tell.

At the other end there is a long tail of predicates that occur once. An agent given a question generates filters from the question, not from a catalogue of saved reports, so the predicate distribution has a far heavier tail than a dashboard-driven workload. Partitioning and clustering schemes tuned by observing recurring predicates are tuned for a distribution the workload no longer has, and the tail pays full scan cost.

Between the two sits the phase that turns out to dominate: sampling. `SELECT * FROM t LIMIT n` is unselective by construction, discards its own result, and is issued by every agent on every cold start. It is the cheapest-looking query in the workload and Section 6.1 finds it accounts for roughly 36% of the platform's scan cost.

There is a measurement consequence worth stating separately, because it affects how these mechanisms are tuned in practice. Sampling traffic is highly repetitive and therefore caches extremely well, which inflates the aggregate cache hit rate: Section 6.5 finds that removing sampling drops the exact-text hit rate from 89.2% to 70.0%. A platform team reading a 89% hit rate off an agentic workload and concluding the cache is working is reading a number dominated by its cheapest phase.

## 4.2 Governance under non-deterministic consumers

This is the mismatch with the fewest good answers, and it deserves the section's weight.

Row- and column-level security, data masking, and access audit were built for consumers whose access pattern is a stable property of a known principal. An analyst's queries vary, but the set of things they might legitimately ask for is bounded by their role, and unusual access shows up as an anomaly worth investigating. An agent's access pattern is generated at inference time and is a function of its input. If that input can be influenced — by a document the agent reads, by a prior agent's output, by a user's prompt — then the access pattern can be influenced. Prompt injection stops being a model safety concern and becomes a data access control problem.

The response emerging in practice is to stop exposing raw tables to agents at all, and instead expose a governed semantic model — measures, dimensions, and their definitions — over a protocol the agent introspects and calls, with the Model Context Protocol currently filling that role. We describe this as an industry pattern rather than an established architectural result, because that is what it currently is. Framed generously, this is a good idea for a second reason: it fixes metric consistency, because an agent pointed at raw tables re-derives joins, grain and metric logic on every prompt and can return different answers to the same question on different runs.

But the security argument for this boundary is weaker than it is usually presented, and the distinction matters because the boundary is increasingly offered as though it settled the question.

What the semantic layer constrains is *what* can be asked. What it does not constrain is *why* it is being asked. An agent restricted to governed measures has a smaller vocabulary than one with raw table access, but every word in that vocabulary is still available to it regardless of whether the question it is answering came from its operator or from text it ingested. Aggregate queries compose: a sequence of individually legitimate, individually-authorised measure requests, each sliced slightly differently, reconstructs membership of the underlying set with arbitrary precision. This is not a novel attack — it is the classic statistical database inference problem — but it acquires new force when the querying party is a tireless process whose question sequence is derived from attacker-influenceable input.

The boundary is therefore a genuine reduction in attack surface and not a closure of it. Our position is that it should be adopted, argued for on the grounds where it is strong — metric consistency, governance legibility, a single place to enforce policy — and not relied upon as an answer to injection. Section 6.4 quantifies what it costs in capability, and Section 7 states the control we think is actually missing.

We report no empirical result on this point: the harness measures cost, not security, and we did not attempt to construct exfiltration sequences against it. That is a limitation rather than a finding, and Section 8 lists it as such.

## 4.3 Colocated agent state

Agents read, and agents also write. Conversation memory, task checkpoints, intermediate results and execution traces are all persistent state that an agent fleet produces continuously, and all of it has the shape analytical infrastructure handles worst: small records, written frequently, read back individually, latency-sensitive on both sides. Meanwhile the same fleet's reads want exactly what a lakehouse provides — large scans over columnar files.

This is the OLTP/OLAP divide arriving through a new door. It has a settled answer in conventional architectures, which is to run separate systems and accept the seam. What is different here is that the two workloads now belong to the *same* logical application, operate over overlapping data, and must agree — a tension also visible in federated enterprise data-platform architectures [12]: an agent whose checkpoint says it has processed a partition needs that claim to be consistent with what the analytical side believes.

Three arrangements appear in practice, each with a distinct failure mode. Writing agent state into the lakehouse itself produces small-file and manifest pressure — exactly the condition table formats spend their compaction machinery fighting — and puts latency-sensitive writes behind commit protocols designed for throughput. Running a separate OLTP store alongside solves the write shape and reintroduces the consistency seam plus a second system to operate. Using a NoSQL store for agent memory, the most common choice in the field, is really the second option with the consistency question answered by declining to ask it, which is defensible for conversation memory and not for checkpoints that gate analytical work.

We contribute nothing empirical on this point: the harness models agent reads only, and extending it to model agent writes is real work rather than a parameter change. We raise it because it is the mismatch with the least published treatment, and because the choice is being made by default in most deployments rather than by analysis.

## 5. Artifact

`agentlake` is released with this paper under a permissive licence. (*Repository URL to be inserted at camera-ready; the artifact accompanies the submission.*)

The harness is ~1,600 lines of Python with one dependency (DuckDB) and a suite of 27 tests. The tests are part of the contribution rather than hygiene: the central claim that correction-phase queries *should* be cache hits is only meaningful if the mutations really preserve meaning, so the suite executes every mutation of every corpus question and compares result sets. Two substantive bugs reported in Section 3 — the over-colliding plan fingerprint and the alias rewriter that silently failed on quoted identifiers — were found by those tests rather than by inspection, as was the shared random stream that invalidated the first mitigation ablation.

The harness ships two drivers. The serial driver interleaves agents deterministically and executes one query at a time, which isolates the work a fleet creates and is exactly reproducible. The concurrent driver runs agents through a thread pool against a shared database with a bounded worker and memory budget, using one cursor per thread, so queries execute in parallel and compete for real resources; this is what produces the latency result in Finding 4. Reporting both is deliberate, since they answer different questions and the serial numbers are the reproducible ones.

To target another engine, implement the `EngineAdapter` interface; the report contract is unchanged. A distributed adapter is the obvious next contribution, for the reason given in Section 7.

## 6. Mitigations

We evaluate five interventions against the reference workload, each expressed as a change to the *workload* rather than to the engine, because that is what the harness can vary honestly. All runs use the same dataset, fleet size (8), total turns (192), and seed; only the named intervention differs. The baseline here is therefore a different configuration from the fleet-size sweep of Section 3.3 and the concurrency sweep of Finding 4, and its absolute scan figure differs accordingly; comparisons are within this table. Independent random streams per decision type ensure that changing one phase's parameters leaves the others' decisions bit-identical. This control matters more than it sounds: an earlier version shared one stream, and disabling discovery — which removes no scan work at all — appeared to make the workload 6% more expensive, purely by shifting every downstream retry and abandonment decision.

| Intervention            | Scan/question | vs. base      | Fan-out | Answered | Question space retained |
|-------------------------|---------------|---------------|---------|----------|-------------------------|
| None (baseline)         | 997,219       | —             | 4.01    | 165      | 100%                    |
| Pinned schema context   | 997,219       | 0.0%          | 4.01    | 165      | 100%                    |
| Cached column profiles  | 639,514       | <b>−35.9%</b> | 1.68    | 165      | 100%                    |
| Tight correction budget | 1,031,613     | <b>+3.4%</b>  | 4.28    | 149      | 100%                    |
| Semantic layer boundary | 665,695       | −33.2%        | 1.67    | 164      | <b>15%</b>              |
| Semantic layer + budget | 673,400       | −32.5%        | 1.70    | 149      | 15%                     |

### 6.1 Cached column profiles are the single best intervention

Serving cardinality and value distributions from a maintained profile, instead of letting each agent sample tables itself, removes 35.9% of rows scanned per answered question and collapses fan-out from 4.01 queries to 1.68 — while answering exactly the same number of questions. It is the only intervention we tested that is close to free.

The finding behind it is that sampling, not analytical querying, is where agent workloads spend their platform budget. A `SELECT * ... LIMIT 100` looks trivial and is not: it is unselective, frequently unpartitioned, and is issued on every cold start by every agent. The existing benchmark literature does not see this cost at all, because it measures the analytical query an agent finally produces.

The capability price is real but modest: profiles go stale, and an agent that cannot inspect actual values is weaker on free-text and high-cardinality columns. We have no production data on how profile staleness behaves in practice, and we note it as the most obvious place where deployment experience would extend this result.

### 6.2 Pinned schema context is the right fix for a different problem

Shipping schema and column semantics with the agent — the direction pursued by work on agentic schema refinement [14] and on supplying agents with an organisation's tribal knowledge [15] — eliminates catalog traffic entirely — 1,152 requests to zero — and changes rows scanned by exactly nothing.

This is worth stating plainly because it is the intervention most often proposed as a cost measure, and against a scan-bound platform it saves nothing. It is nevertheless the correct intervention for a catalog-bound one, and catalog request volume is precisely what a shared REST catalog serving a growing fleet has to absorb. The lesson is that "agent platform cost" is not one quantity, and an intervention that helps a metadata service may be irrelevant to an execution engine.

### 6.3 Tightening the correction budget is counterproductive

Capping retries at one made the workload **3.4% more expensive per answered question**, not cheaper. We expected a saving and found the opposite.

The mechanism is visible in the table: answered questions fall from 165 to 149 while fan-out *rises* from 4.01 to 4.28. A capped agent still issues its failed first attempt and still pays for it; what the cap removes is the retry that would have converted that sunk cost into an answer. Work already spent is not recovered by refusing to finish.

This generalises to a warning about a whole class of mitigation. Interventions that cap, throttle or rate-limit agent activity look attractive on a per-query dashboard — correction-budget capping does reduce total queries issued — but the unit of value is an answered question, and any intervention evaluated against the wrong denominator can be worse than nothing. This is Finding 1 reappearing as an operational trap rather than a measurement one.

#### 6.4 The semantic layer boundary has a capability cliff

Exposing governed measures and dimensions rather than raw tables removes 33.2% of scan cost and all catalog traffic, at a price we can now quantify: **15% of the question space remains answerable**. Restricting agents to modelled metrics means questions nobody modelled cannot be asked at all.

Two observations follow. First, the saving is slightly *worse* than cached profiles alone (−33.2% vs −35.9%) while costing 85% of the question space, so on cost grounds alone the semantic layer is not the better trade. Its justification is governance and metric consistency — Section 4.2 — and it should be argued on those terms rather than as an efficiency measure. Second, combining it with a correction budget is worse than either sensible intervention alone (−32.5%, 149 answered, 15% answerable), because the mitigations compose their costs without composing their benefits.

Our 15% is a property of how we construct the curated space — the unparameterised base corpus against the full parameterised one — and is emphatically not a measurement of any organisation's semantic model coverage. The transferable result is the shape of the trade-off and the fact that it is steep, not the number. How coverage actually moves over time in a production semantic layer is something we cannot observe and would want to know.

#### 6.5 Plan-keyed caching, and why reported hit rates flatter

Keying the result cache on the optimised plan rather than on query text raises the hit rate from 89.2% to 93.6% and would avoid 8.0% more scanned rows (135.8M vs 125.7M). We report this separately from the table above because it is a different kind of claim: the harness simulates both caches as observers over the real query stream and does not skip execution on a hit, so these are counterfactual savings, whereas the workload interventions genuinely do not run.

The ablation also exposes something about cache metrics themselves. Removing sampling traffic drops the exact-text hit rate from 89.2% to 70.0%. The baseline's healthy-looking hit rate is substantially an artifact of highly repetitive `SELECT * ... LIMIT` queries; on analytical queries alone the cache misses nearly a third of the time. Any agentic deployment reporting a cache hit rate over undifferentiated agent traffic is reporting a number inflated by its cheapest phase.

#### 6.6 What we could not evaluate

Two mitigations we consider promising are outside what this harness can honestly test. **Small-model routing** [20] — serving discovery and sampling from a cheap model and reserving a frontier model for candidate generation — targets inference cost, which we do not measure, and our synthetic agents have no model in the loop. **Phase-aware admission control** — scheduling discovery traffic differently from analytical traffic — requires a real scheduler and a real queue; an in-process engine has neither. Both are noted in Section 7 rather than claimed here.

## 7. Open problems

**1. Caching for semantically near-duplicate queries.** Plan keying is an improvement, not a solution: it recovers 8.0% more scanned rows and still misses roughly 30% of analytical queries once sampling is excluded. Exact-match caching is the wrong primitive for a consumer that never phrases the same question the same way twice, and plan-equality is merely a less wrong one — it still misses queries that differ in ways that do not change the answer the user needs, such as a slightly wider date range. Cache keys based on answer-equivalence rather than plan-equality are an open design problem.

**2. Governance without the capability cliff.** Section 4.2 argues that access control assumes a consumer whose intent is inferable from their identity, and that agents break that assumption. Section 6.4 shows the cost of the current answer: the semantic layer narrows the attack surface at the price of 85% of the question space, and it still does not close the exfiltration path, since an injected agent restricted to governed measures can compose a sequence of individually legitimate aggregates. What is needed is a control that binds data access to *task provenance* rather than to principal identity — something that can distinguish an agent legitimately answering the question it was asked from the same agent answering a question that was injected into it. We do not know of a deployed system that does this.

**3. How far does the latency wall move on a distributed engine?** Finding 4 establishes that contention appears as soon as a shared resource pool exists: under real concurrent execution against a bounded budget, work stays additive while mean latency scales as  $\text{fleet}^{0.76}$  and throughput saturates at four agents. What we cannot say is where that wall sits on a real cluster. A distributed engine adds contention points we do not model — a REST catalog serving metadata to the whole fleet, shuffle competing for network, a scheduler making cross-node placement decisions — and removes others by adding capacity. The experiment is the same sweep against Trino or Spark with a catalog under load. Work on scheduling and serving agentic LLM workloads [16]-[18] addresses the inference side of the same deployment and would compose with it. We would expect the exponent to be worse and the saturation point to be higher, and we would not bet much on either.

**4. Cost models where the unit of value is not the unit of cost.** Finding 1 and Section 6.3 are the same problem seen twice: an answered question is what an organisation buys, an issued query is what it pays for, and the ratio is neither stable nor observable from per-query telemetry. Admission control, chargeback, capacity planning and auto-scaling are all currently built on the wrong denominator. Attributing platform cost to a *task* rather than a query, across a fan-out the platform cannot see the boundaries of, is unsolved and is a prerequisite for most of the operational tooling this workload needs.

**5. Calibration, and the absence of public traces.** Every phase parameter in this paper is a declared assumption. The magnitudes we report are therefore properties of a model, and only the structure transfers. There is no public corpus of real multi-agent analytical traces, and the ones that exist inside organisations are not shareable in raw form. An anonymised, aggregated trace format — phase-labelled query counts, fan-out distributions, correction and abandonment rates, with no SQL text or schema — would be enough to calibrate work like this, and would cost far less to release than a full query log. We would rather see that standard than another benchmark.

**6. Storage layouts for colocated agent state.** Section 4.3 describes agent memory, checkpoints and traces as an OLTP-shaped write workload landing beside an OLAP-shaped read workload. The harness does not model agent writes at all, so we contribute nothing empirical here, and it remains the mismatch with the least published work: small frequent latency-sensitive writes, analytical reads over the same logical dataset, and a table format whose manifest and compaction machinery was designed for neither.

## 8. Threats to validity

We state these in the order a sceptical reviewer would raise them.

**The agents are synthetic.** No model is in the loop. A turn is generated from an explicit phase model and is fully determined by a seed. This buys reproducibility and the ability to vary one parameter at a time; it costs realism in the one place that matters most, which is whether real agents behave as our phase model says. We consider the *ordering* of the results robust to this — sampling dominating scan cost, corrections defeating text-keyed caches, fan-out separating per-query from per-question cost — because each follows from the structure of the agent loop rather than from a parameter value. We consider the *magnitudes* to be properties of the model, and we have tried never to state them otherwise.

**Every phase parameter is uncalibrated.** The harness records this in the model itself: a `PhaseModel` carries a provenance string and reports `is_calibrated`, and the defaults declare themselves uncalibrated. A reader who believes our correction rate is twice too high should divide accordingly; the harness makes that a one-line change and a re-run.

**The engine is in-process.** DuckDB gives us a real shared buffer pool, a real worker pool and, through the concurrent driver, real contention — which is what Finding 4 needed. It does not give us network shuffle, a distributed catalog under load, or cross-node scheduling. Finding 4's latency result should therefore be read as establishing that contention appears once a shared resource pool exists, not as predicting its magnitude on Trino or Spark. We would expect a distributed engine to be worse, and we have not shown it.

**Caches are simulated.** Both cache models observe the real query stream but do not skip execution on a hit, so cache savings are counterfactual estimates. This is stated at the point of use in Section 6.5. The comparison *between* the two keying strategies is sound, since both observe the identical stream; the absolute savings are estimates.

**The schema is one star, and the corpus is small.** Six base questions expanded parameterically to thirty-nine. Conventional aggregations over a clean star schema are the friendly case. A wide, messy, real schema would plausibly increase discovery and sampling cost — the phases we already find dominant — so we expect this to bias our results toward understating the effect, but we have not shown that either.

`bytes_read` is unreliable in this configuration and we do not report it; `rows_scanned` is used throughout.

**We measure no security property.** Section 4.2 argues that the semantic-layer boundary is weaker than commonly presented. That argument is analytical. We did not construct exfiltration sequences and we report no empirical security result.

**One positioning source was not read in full.** SQLStorm [1], AgenticDataBench [3], FDABench [4] and the survey [22] were read from their released artifacts, and the counts we attribute to [3] and [4] were recomputed from those artifacts rather than quoted. Text-to-Big SQL [2] has no artifact we could locate and is characterized from its abstract and published summaries; its figures are attributed rather than asserted. Reading [1] and [4] changed the paper materially — an earlier draft described SQLStorm as an agent-accuracy benchmark, which it is not, and did not account for FDABench's phase instrumentation — which is a reason to treat the remaining unread source as a live risk rather than a formality.

**We measure no inference cost.** The agent-side cost of running the models is outside the harness entirely. A complete account of agentic workload economics would include it, and Section 6.6 notes the mitigation we consequently could not evaluate.

---

## 9. Conclusion

Analytical platforms are being asked to serve a consumer they were not designed for, and the first problem is not that the consumer is expensive. It is that the instruments do not report it as expensive. Agent queries scan fewer rows each than the human workload they displace, while the workload as a whole costs more than twice as much per question answered; a per-query dashboard shows an improvement throughout. Everything downstream of that — capacity planning, chargeback, throttling policy — is being decided on the wrong denominator, and Section 6.3 shows a common throttling reflex making things actively worse for exactly that reason.

The second result is that this workload scales in two ways at once. Rows scanned per answered question is flat in fleet size; mean query latency scales as  $\text{fleet}^{0.76}$  and throughput saturates at four concurrent agents. An organisation watching cost will see linear growth and conclude nothing is wrong, while the agents themselves become progressively less usable. These are not competing measurements of one phenomenon. They are two phenomena, and they want different interventions — a budget and an admission controller. Read alongside the text-to-Big SQL result that execution overtakes agent interaction as data grows, the picture is that the execution side becomes the binding constraint along two independent axes: data volume, and how many agents share the engine.

The third is that the mitigation which helps most is the one least discussed. Serving cached column profiles instead of letting each agent sample tables for itself removes 35.9% of scan cost at no cost in answered questions, because sampling — not analytical querying — is where this workload actually spends the platform. The interventions that receive more attention do less: pinned schema context saves no scan work at all, and the semantic-layer boundary buys a comparable saving at the price of 85% of the question space, which is a reason to argue for it on governance grounds rather than efficiency ones.

None of this competes with the agent-side literature, and we want to be precise about the relationship rather than claim more territory than we hold. FDABench already decomposes an agent's latency and tokens into decision, execute, retry and generate phases; that is the same shape of accounting we apply, pointed at the other side of the interface. The two belong together. An organisation that knows what its agents spend and what its platform spends can reason about the whole cost of an answered question; today it can usually see neither, and the half that is easier to instrument is the half that misleads.

We have tried to be as careful about what we did not establish as about what we did. Three of the results here are corrected versions of artifacts we first mistook for findings: a spurious negative scaling exponent produced by letting total work grow with fleet size, a plan fingerprint coarse enough to collide distinct questions and inflate its own cache hit rate, and a shared random stream that made an intervention removing no work appear to cost 6%. Two further corrections were to our reading of other people's work, and surfaced only when we stopped working from abstracts and read the released artifacts: we had described an engine-benchmarking system as an agent-accuracy benchmark, and had overlooked the phase instrumentation that makes FDABench the closest existing relative of our taxonomy. Each of the first three was caught by the harness rather than by inspection, which is the argument for releasing it; the last two are the argument for reading the code that ships with a paper. We expect both to catch things we have not thought of, including in our own results.

---

## 10. References

Provenance is stated per entry. Counts attributed to an artifact were recomputed from it.

### Read from released artifacts

1. T. Schmidt, V. Leis, P. Boncz, T. Neumann. *SQLStorm: Taking Database Benchmarking into the LLM Era*. PVLDB 18(11): 4144–4157, 2025. doi:10.14778/3749646.3749683. Artifact: [github.com/SQL-Storm/SQLStorm](https://github.com/SQL-Storm/SQLStorm). (Query counts, dataset scales, engine list and the dialect-rewriting step are taken from the released benchmark and its documentation.)
2. *AgenticDataBench: A Comprehensive Benchmark for Data Agents*. arXiv:2607.01647. Tsinghua University and Ant Digital Technologies. Artifact: [github.com/AgenticDataBench/AgenticDataBench](https://github.com/AgenticDataBench/AgenticDataBench). (433 skills, 6,510 source solutions, 344 tasks, 246 public gold tasks and 15 domains were recomputed from the repository's data files.)
3. Z. Wang, S. Zhang, H. Yuan, J. Zhu, W. Dong, G. Cong. *FDABench: A Benchmark for Data Agents on Analytical Queries over Heterogeneous Data*. arXiv:2509.02473. KDD 2026. Nanyang Technological University, National University of Singapore, Huawei. Artifact: [github.com/fdabench/FDABench](https://github.com/fdabench/FDABench). (Task count and task types from the repository; the decision/execute/retry/generate phase instrumentation and token tracking are read from its agent source.)
4. Y. Zhu, L. Wang, C. Yang, X. Lin, B. Li, W. Zhou, X. Liu, Z. Peng, T. Luo, Y. Li, C. Chai, C. Chen, S. Di, J. Fan, J. Sun, N. Tang, F. Tsung, J. Wang, C. Wu, Y. Xu, S. Zhang, Y. Zhang, X. Zhou, G. Li, Y. Luo. *A Survey of Data Agents: Emerging Paradigm or Overstated Hype?* Companion repository: [github.com/HKUSTDial/awesome-data-agents](https://github.com/HKUSTDial/awesome-data-agents). (Read in full; the term-frequency table in Section 2.3 is computed over its text and is reproducible with `harness/analyse_survey_coverage.py`.)

### Characterized from abstract and published summaries

1. G. T. Eizaguirre, L. Tissen, M. Sánchez-Artigas. *Both Ends Count! Just How Good are LLM Agents at Text-to-"Big SQL"?* arXiv:2602.21480; ACM DL doi:10.1145/3805621.3807640. Universitat Rovira i Virgili and RWTH Aachen. **No released artifact located; not read in full.** The "both ends" characterization and the TPC-H Q21 SF 10 → 1000 figure are taken from published summaries of the paper and must be confirmed against its text.

### Cited without individual characterization

1. *Agent Bain vs. Agent McKinsey: A New Text-to-SQL Benchmark for the Business Domain*. arXiv:2510.07309.
2. *ReViSQL: Achieving Human-Level Text-to-SQL*. arXiv:2603.20004.
3. *AgentNLQ: A General-Purpose Agent for Natural Language to SQL*. arXiv:2605.19010.
4. *MARS-SQL: A Multi-Agent Reinforcement Learning Framework for Text-to-SQL*. arXiv:2511.01008.
5. *DS-STAR: Data Science Agent for Solving Diverse Tasks across Heterogeneous Formats and Open-Ended Queries*. arXiv:2509.21825.
6. *Beyond the Data Mesh Illusion: Designing Modern AI-augmented Lakehouses*. arXiv:2605.27131.
7. *What Went Wrong with Data Lakes? A 15-Year Reality Check from the Field*. arXiv:2606.08266.
8. *Enterprise Data Science Platform: A Unified Architecture for Federated Data Access*. arXiv:2512.03401.
9. *Democratizing Cloud Data Lake Analytics: Natural Language Access to Apache Iceberg via LLM Agents*. Frontiers in Big Data, doi:10.3389/fdata.2026.1785710.
10. *Towards Agentic Schema Refinement*. arXiv:2412.07786.
11. *Arming Data Agents with Tribal Knowledge*. arXiv:2602.13521.
12. *HexAGenT: Efficient Agentic LLM Serving via Workflow- and Heterogeneity-Aware Scheduling*. arXiv:2605.16637.
13. *Cloud-native and Distributed Systems for Efficient and Scalable Large Language Models — A Research Agenda*. arXiv:2604.17227.
14. *Trade-offs in Decentralized Agentic AI Discovery Across the Compute Continuum*. arXiv:2605.11839.
15. *A Survey of LLM-Driven AI Agent Communication: Protocols, Security Risks, and Defense Countermeasures*. arXiv:2506.19676.

16. *Small Language Models are the Future of Agentic AI*. arXiv:2506.02153.
17. Further data-agent benchmarks surveyed in [22]: *DataSpace* (arXiv:2608.03451), *DAComp* (arXiv:2512.04324, ICLR 2026), *KramaBench* (arXiv:2506.06541, ICLR 2026), *CoDA-Bench* (arXiv:2606.15300, ICML 2026), *DABstep* (arXiv:2506.23719), *Can AI Agents Answer Your Data Questions?* (arXiv:2603.20576).